22. File Handling (I/O): Reading and Writing Data

Created by suncaper, last modified about 3 hours ago

Introduction

In the previous lesson, we learned how to protect our pipelines from crashing using Exception Handling. But where is all this data actually coming from? So far, we have been manually typing lists and dictionaries directly into our scripts. In the real world, data lives in files.

File Handling (often called **I/O**, which stands for Input/Output) is the process of using Python to open, read, create, and modify external files like CSVs, JSONs, and plain text logs.

In Data Engineering, I/O represents the "Extract" and "Load" in an ETL pipeline. You extract data by reading a source file into Python's memory, you transform it, and you load it by writing the results into a brand new file.

In this lesson, we will cover:

- Opening files and the three main modes: Read (`'r'`), Write (`'w'`), and Append (`'a'`).
- The danger of leaving files open in memory.
- The industry standard: Using the `with open(...)` context manager.
- Why raw Python I/O is still critical in the age of Spark and Pandas.

Part 1: The Basics (`open`, `read`, `write`, `close`)

To interact with a file, Python needs to unlock it using the `open()` function. You must tell Python the file's path and the **Mode** you want to use.

- **`'r'` (Read):** Opens a file to read its contents. (Fails if the file doesn't exist).
- **`'w'` (Write):** Opens a file to write data. **Warning:** This completely overwrites any existing data! If the file doesn't exist, it creates a new one.
- **`'a'` (Append):** Adds new data to the very end of an existing file without deleting what is already there.

1. The Manual Way (Not Recommended)

```
# Open a file in Write mode
log_file = open("pipeline_log.txt", "w")

# Write a string to the file
log_file.write("ETL Job Started Successfully.\n")

# ❌ DANGER: You MUST remember to close the file!
log_file.close()
```

If you forget to call `.close()`, the file remains locked in your computer's memory. If your pipeline does this to 10,000 files, your server will crash with a "Memory Leak" or "Too Many Open Files" error.

Part 2: The Context Manager (`with open`)

Because forgetting to close files is such a common cause of pipeline failures, Python introduced the **Context Manager**.

By using the `with open(...) as variable:` syntax, Python creates an indented block of code. The moment your code exits that indented block, **Python automatically closes the file for you**, even if your script crashes due to an error!

(Think of this as an automatic `finally` block for files).

```
# ✅ The Industry Standard Way
with open("pipeline_log.txt", "a") as my_file:
    # We are inside the context. The file is open.
    my_file.write("Appending a new record...\n")

# We are outside the indent. The file is already safely closed!
```

Reading Data with Context Managers:

```
# Reading the file we just created
with open("pipeline_log.txt", "r") as my_file:
    # .read() grabs the entire file as a single massive string
    content = my_file.read()
    print(content)
```

Part 3: The "Big Data" Context (Why do we care?)

If Apache Spark has built-in commands like `spark.read.csv()`, why do Data Engineers need to know basic Python I/O?

1. **Configuration Files:** Massive Spark pipelines are usually driven by small `.json` or `.ini` configuration files containing database passwords and target folders. Spark doesn't read these; pure Python `with open()` reads them before Spark even boots up.
2. **API Interactions:** When you pull data from a REST API (like Salesforce or Twitter), you often need to save that raw JSON response to a local temporary file before moving it to cloud storage (like AWS S3). You use Python's `write()` for this.
3. **Custom Logging:** While big frameworks have their own logging, you will frequently write custom `.txt` log files to record exactly which batches succeeded or failed during a complex orchestration script.

Part 4: Hands-on Exercise (Simulating an ETL Output)

Let's simulate the final step of an ETL pipeline (The "Load" phase). We have a list of cleaned user IDs. We need to write them into a brand new CSV file, and then read that file back to verify the data was saved correctly.

1. Open PyCharm.
2. Create a new file named `io_practice.py`.
3. Type the following code:

```
# ---------------------------------------------------------
# Big Data Simulation: Writing and Reading a CSV
# ---------------------------------------------------------

"""
Scenario: Our transformation is complete. We have a list of clean IDs.
We need to WRITE them to a file, and then READ the file to verify.
"""

# 1. THE DATA
clean_user_ids = ["user_101", "user_102", "user_103"]
file_name = "target_users.csv"

print("--- Step 1: Writing Data (Load) ---")
```

```
# 2. WRITE: Open in 'w' mode to create the file
with open(file_name, "w") as target_file:
    # Write a header row first
    target_file.write("user_id\n")

    # Loop through our clean data and write each row
    for uid in clean_user_ids:
        target_file.write(uid + "\n") # \n adds a new line

print("SUCCESS: File saved and automatically closed.")
print("\n--- Step 2: Reading Data (Verification) ---")

# 3. READ: Open in 'r' mode to read the file we just made
with open(file_name, "r") as source_file:
    # .readlines() reads the file and returns a List of strings (one per line)
    saved_lines = source_file.readlines()

    for line in saved_lines:
        # We use .strip() to remove the hidden \n characters for clean printing
        print("Found in file: " + line.strip())

print("----------------------------------------")
print("Pipeline verified and complete.")


Run the code. Output:

Plaintext

--- Step 1: Writing Data (Load) ---
SUCCESS: File saved and automatically closed.

--- Step 2: Reading Data (Verification) ---
Found in file: user_id
Found in file: user_101
Found in file: user_102
Found in file: user_103
----------------------------------------
Pipeline verified and complete.
```

What happens?

- We use `with open(..., "w")` to safely create a new file called `target_users.csv`.
- We iterate through our Python list, using `.write()` to append each ID and a newline character (`\n`).
- The block ends, and Python automatically closes and saves the file to your hard drive.
- We use `with open(..., "r")` to reopen the file, using `.readlines()` to convert the text document back into a Python List.
- We loop through that new list, strip out the invisible formatting characters, and print the verified results.

Summary

- **I/O (Input/Output)** is how Python interacts with your hard drive.
- **Modes:** Use `'r'` to read, `'w'` to overwrite/create, and `'a'` to append new data to the end.
- **`with open(...):`** The industry-standard Context Manager. It guarantees that your file is safely closed, preventing catastrophic memory leaks.
- **Data Pipelines depend on I/O:** While Big Data frameworks handle massive datasets, standard Python File Handling is mandatory for reading pipeline configurations, saving API payloads, and generating custom execution logs.

No labels